

Learning convolutional neural network

Example of convnet of dimension

$$(32 \times 32) \times 3 = 1024 \times 3 = 3072 \text{ total}$$

↑
spatial grid

1024 positions

3 values per pixel:
red, green, blue

Key components of a CNN

1. Input layer

- Stores the values of the pixels of image ($32 \times 32 \times 3$)
- Preserve the spatial structure of image to extract more feature

↓ RGB images

age, -

2. convolutional layer

1	0	1	0	1	0
0	1	1	0	1	1
1	0	1	0	1	0
1	0	1	1	1	0
0	1	1	0	1	1
1	0	1	0	1	0

x_{11}	x_{12}	x_{13}
1	0	1
x_{21}	x_{22}	x_{23}
0	1	1
x_{31}	x_{32}	x_{33}
1	0	1

Image patch
(Local receptive field)

bitwise operation
 $\sum_{i=1}^m \sum_{j=1}^m x_{ij} \cdot y_{ij}$

y_{11}	y_{12}	y_{13}
1	2	3
y_{21}	y_{22}	y_{23}
4	5	6
y_{31}	y_{32}	y_{33}
7	8	9

Kernel (filter)

↓

Output

31		

Key components of a convolution Layer

1. Filters (kernel)

- small matrices that extract specific feature from input
- They are updated during the process of training

2. Stride

- step size of which filter moves across data so step size ↑ → output dimensions ↓

3. Padding

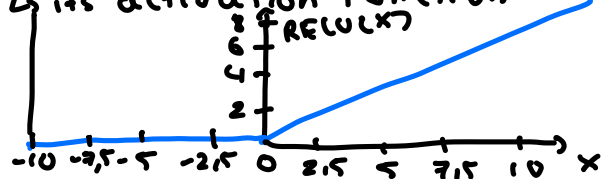
- Zero or other values can be added to the input to control the spatial dimensions of output

4. activation Function

After convolution, a non linear function is applied to let the network learn complex relationship in data

ReLU, most popular

↳ its activation Function



Why choose ReLU?

- **Simplicity** since it's a thresholding operation
- **non-linearity**, even if it looks like a piece-wise function, it's still a non-linear function. So this allows the model to learn more complex data patterns and have a better understanding between features
- **sparse activation**: ReLU's ability to output 0 for negative inputs, so not all neurons are activated at the same $\rightarrow$ faster and efficient computation
- **gradient computation**: better for backpropagation as derivative is simple

0 when input ≤ 0

1 when input > 0

helps for vanishing gradient problem

$\hookrightarrow$ common issue with sigmoid or tanh activation functions

Types of activation Functions

Activation function	Formula	Output range	Advantages	Disadvantages	Use cases
 ReLU	$f(x) = \max(0, x)$	$[0, \infty)$	- simple - efficient - reduce vanishing problem - sparse activation	- Dying ReLU problem - Unbounded output	Hidden layers in deep networks
 Leaky ReLU $\alpha = 0.1$	$f(x) = \begin{cases} x & x > 0 \\ \alpha x & x < 0 \end{cases}$	$(-\infty, \infty)$	- prevents dying ReLU - Allow small gradient for negative values	Require manual tuning of α	(alternative)
 PReLU $\alpha = 0.1$	Same as Leaky ReLU	$(-\infty, \infty)$	- learns optimal slope - Better performance than ReLU in some cases	- Risk of overfitting - More parameters	Deep networks where ReLU fails
 Sigmoid	$f(x) = \frac{1}{1 + e^{-x}}$	$(0, 1)$	- smooth output - good for probabilities	- vanishing gradient - Not zero-centered	Output layers (binary classification problem)
 Tanh	$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$(-1, 1)$	- zero-centered output - Better than sigmoid	vanishing gradient	Hidden layers (normalized data)
 ELU	$f(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$	$(-\alpha, \infty)$	- smooth for negative numbers - Faster convergence	slower computation than ReLU	Deep networks

3. Activation Layer

it introduces non linearity by applying an element-wise to the output of the convolutional layer

$\hookrightarrow$ helps learn complex patterns beyond linear relationships

• Output dimensions stays the same

4. Pooling layer

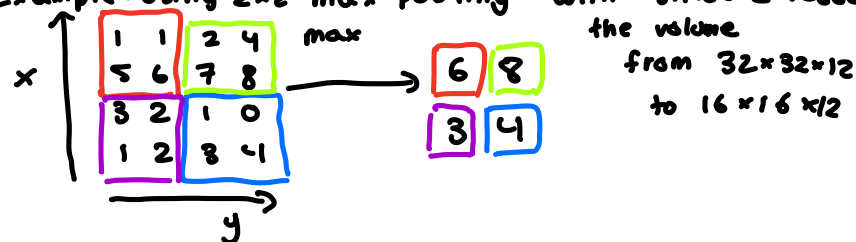
used to reduce spatial dimensions of feature maps

→ makes computation faster, reducing memory usage and helps prevent overfitting. Inserted typically inside convolutional layers in a CNN.

- Max pooling
- Average pooling
- Reduces width

And height while keeping depth unchanged

Example: using 2×2 max pooling with stride 2 reduces



5. Flattening

it converts multi-dimensional feature maps into a one-dimensional vector after convolution and pooling. This vector is then passed to the fully connected layer for classification or regression

Ex.: Flattening a $16 \times 16 \times 12$ results in a vector of (3072) size

6. fully connected layer

↳ performs high-level reasoning using extracted features and produces the final classification scores.

Ex.: The 3072-length vector is connected to neurons for classification

7. Output Layer

converts final scores into probabilities using activation functions like Sigmoid (binary classification) or Softmax (multi-class classification)

Ex.: For 10 classes, Softmax produces 10 probability values each representing the likelihood of the class.

Regularization in Machine learning

adding a penalty to prevent overfitting and perform better on unseen data



Types of Regularization

1. Lasso Regression (L1)

adds absolute value of magnitude of the coefficient as a penalty term to the Loss function (L.).

L_1 can help shrink some coefficients to 0 which helps selecting important features and ignoring less important ones

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m |w_j| \quad \hat{w} = (X^T X)^{-1} X^T y$$

where

m : number of features

n : number of examples

y_i : actual target value

$\hat{y}_i$: predicted target

2. Ridge regression (L2)

adds squared magnitude to loss function. Handles multicollinearity shrinking coefficients of correlated features reducing variance and prevent any single feature from dominating the model

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m w_j^2$$

3. Elastic Net Regression

combines L_1 and L_2 with an extra parameter that controls L_1 and L_2 regularization

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left((1 - \alpha) \sum_{j=1}^m |w_j| + \alpha \sum_{j=1}^m w_j^2 \right)$$

α : Mixing parameter where $0 \leq \alpha \leq 1$

and $\alpha = 1 \rightarrow L_1$

$\alpha = 0 \rightarrow L_2$

in between balance between both of them

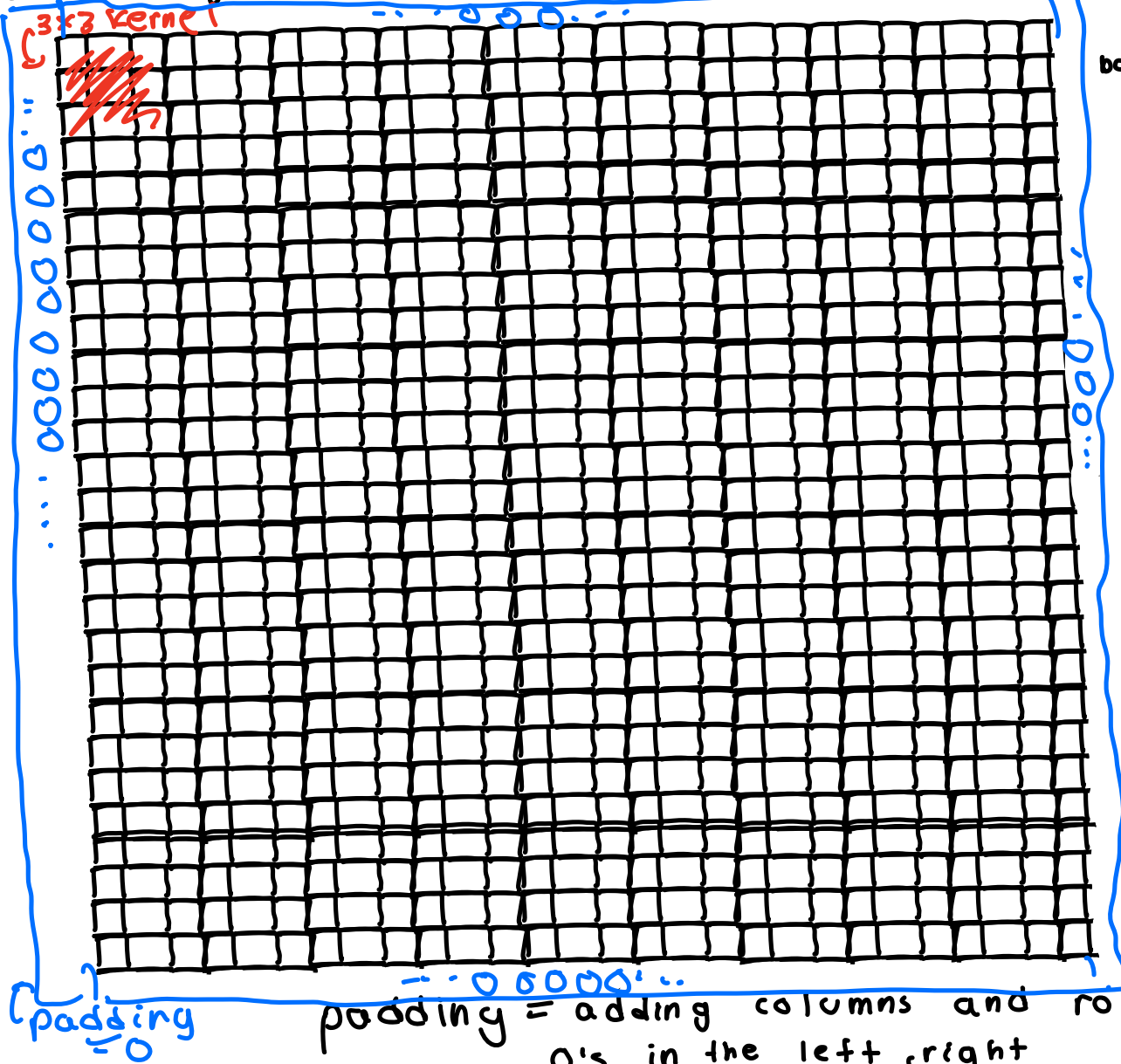
nn.Conv2d (in channels, out channels, kernel, padding)

nn.MaxPool2d (kernel size) $\rightarrow$ Pooling layer

nn.Dropout (%) $\rightarrow$ during training only,
randomly zeroes 25% of
the values every forward pass

One image 28×28 each cell takes value between

0 and 1
based on the
proportion
of the
cell



if we have a

3×3 kernel, 28×28 image input

with padding = 1

then

the number of kernels that fits = 28×28

so if we have output of 32

then we have $32 \times 28 \times 28$ values in total

$$\text{Entropy} = -p_i \log_b(p_i)$$

Cross Entropy loss

$$= - \sum_{i=1}^n Y_i \log_b(p_i)$$

criterion is nn. (Cross Entropy Loss)

output = model(data)

implicitly call model.forward(data)

Back propagation

in neural network

algorithm that trains nn by reducing prediction error
it computes gradients using chain rule to update weights
and biases to improve performance

